

*wix10beF50a398c

No.	Time	Source	Destination	Protocol	Length	Info
2	5.627998986	192.168.29.207	192.168.29.1	ICMP	98	Echo (ping) request id=0x4128, seq=1/256, ttl=64 (reply in 3)
4	6.628964448	192.168.29.207	192.168.29.1	ICMP	98	Echo (ping) request id=0x4128, seq=2/512, ttl=64 (reply in 5)
6	7.630511211	192.168.29.207	192.168.29.1	ICMP	98	Echo (ping) request id=0x4128, seq=3/768, ttl=64 (reply in 7)
8	8.632246033	192.168.29.207	192.168.29.1	ICMP	98	Echo (ping) request id=0x4128, seq=4/1024, ttl=64 (reply in 9)
10	9.633699716	192.168.29.207	192.168.29.1	ICMP	98	Echo (ping) request id=0x4128, seq=5/1280, ttl=64 (reply in 11)
75	33.185655914	192.168.29.207	192.168.29.1	ICMP	71	Destination unreachable (Port unreachable)
2914	121.586682051	192.168.29.207	192.168.29.1	ICMP	71	Destination unreachable (Port unreachable)
3573	297.865046418	192.168.29.208	192.168.29.1	ICMP	71	Destination unreachable (Port unreachable)
3592	312.833507347	192.168.29.208	192.168.29.1	ICMP	98	Echo (ping) request id=0x3a49, seq=1/256, ttl=64 (no response found!)
3593	313.841705517	192.168.29.208	192.168.29.1	ICMP	98	Echo (ping) request id=0x3a49, seq=2/512, ttl=64 (no response found!)
3594	314.865715428	192.168.29.208	192.168.29.1	ICMP	98	Echo (ping) request id=0x3a49, seq=3/768, ttl=64 (no response found!)
3599	315.889701078	192.168.29.208	192.168.29.1	ICMP	98	Echo (ping) request id=0x3a49, seq=4/1024, ttl=64 (no response found!)

Figure 5: Output of Wireshark

This indicates that in the *POST_ROUTING* hook, the packet is dropped and not sent to wire. A log from *dmesg* command is shown in Figure 4 and describes the functionality of the module.

In this example you can also see the use of *NF_DROP* or *NF_ACCEPT* return values. The meaning of these values is self-explanatory. But there are a few more return values as well, which include:

- *NF_REPEAT*: Repeat the hook function.
- *NF_QUEUE*: Queue the packet for user space processing. To implement this in user space, we need to use the libraries *nfnethlink* and *netfilter_queue*.
- *NF_STOLEN*: Further processing of the packet and freeing memory is up to your module.

Packet mangling

Netfilter can also be used for packet mangling or modification. In the same URL (<https://github.com/SupriyoGanguly/Linux-Firewall-by-netfilter>), you can find one more file *Mangle.c*. The corresponding makefile is *Makefile_mangle*.

In this hook, the ICMP ping packet source IP is modified just before sending out the packet. You can see the code below:

```
/* This function to be called by hook. */

static unsigned int main_hook(void *priv, struct sk_buff
*skb, const struct nf_hook_state *state)
{

    struct iphdr *ip_header = (struct iphdr *)skb_
network_header(skb);

    if (ip_header->protocol == IPPROTO_ICMP) {
        printk(KERN_INFO "Mangle icmp packet. %x\n", ip_
header->saddr);
    }
}
```

```
ip_header->saddr = 0xd01da8c0;
}

return NF_ACCEPT;
}
```

The output of Wireshark capture shown in Figure 5 depicts that before loading this module, the ping request to destination IP 192.168.29.1 goes from the original IP of the interface, i.e., 192.168.29.207. But after loading the module, the ping request goes from the modified IP of the interface, i.e., 192.168.29.208. However, the physical interface IP is unchanged.

Compiling the code

To compile and test the downloaded module, just use:

```
$ make

$ sudo insmod firewall
```

To remove it, use the following command:

```
$ sudo rmmod firewall
```

This article is a simple tutorial on building firewall modules using Netfilter. You can also do packet capturing by simply using the *NF_INET_PRE_ROUTING* hook number in this example. You can even use this example to simulate a man-in-the-middle attack for your devices, to test the cybersecurity. **END** 🐧

By: Supriyo Ganguly

The author is an M. Tech in embedded systems, and works as a senior technical officer at the Electronics Corporation of India Limited, Hyderabad.